

Chapter 3. Docker Networking

3.0 Introduction

As you build your distributed application, services that compose it will need to be able to communicate with each other. These services, running in containers, might be on a single host or on multiple hosts and even across data centers. Therefore container networking is a critical enabler of any Docker-based distributed application.

The techniques used for networking containers are very similar to the techniques used to network virtual machines. Containers on a host can be attached to a software switch, and iptables are used to control network traffic between containers and expose processes running in the container on ports of the host.

At installation time, the Docker engine sets up multiple default networking behaviors on your hosts, which gives you a fully working setup from the start. In [Recipe 3.1](#) we start by introducing some Docker commands that let you find the IP addresses of your containers, then we show you how to expose a container port on a host port in [Recipe 3.2](#). In [Recipe 3.3](#) we take a deeper look at container linking, a mechanism to help with service discovery of multiple containers.

Networking is such an important topic for distributed applications that we felt it necessary to dive deeper into the nuts and bolts of it. [Recipe 3.4](#) explains the default Docker bridge setup while [Recipe 3.6](#) shows you how to change the defaults by modifying the Docker engine options. [Recipe 3.8](#) and [Recipe 3.9](#) go even deeper and show you how to create your own network switch and use it for networking your containers. While not necessary, understanding how container networking works and being able to modify it will help you with operating your applications in production.

While container networking is very similar to virtual machine networking, there exists a major difference. With containers you can choose the networking stack being used ([Recipe 3.5](#)). For example, you can share the networking stack of your host with a container, so this allows you to give a container the same IP address than your host. It also allows you to share the same network stack between containers. A lot is possible with container networking, and to explore all the possibilities [Recipe 3.7](#) introduces a nice utility: pipework. Spending a little bit of time with pipework

and trying to understand what it does will greatly enhance your understanding of containers and networking.

So far all the recipes presented have been for a single host; however, in a real distributed application, dozens, hundreds, or even thousands of hosts might be involved. In [Recipe 3.10](#) we give a basic example of creating a tunnel between two hosts to provide a common IP subnet for containers running on two different hosts. This recipe is only shown for training purposes and should not be considered a production solution. Instead, [Recipe 3.11](#), [Recipe 3.12](#), and [Recipe 3.13](#) are recipes that introduce Weave Net and Flannel. They have been contributed by Fintan Ryan and Eugene Yakubovich and introduce production-ready solutions for multihost container networking.

The chapter ends with a peek at Docker network ([Recipe 3.14](#)) and a deep dive into the VXLAN configuration being used ([Recipe 3.15](#)). Docker network is currently under development but should soon be part of the standard Docker release. Solutions like Weave Net, Flannel, and Calico should be usable in Docker network through the plug-in mechanism being developed.

To summarize what you will learn in this chapter, the first few recipes will cover some basic concepts that use the default Docker networking configuration. These should be enough for developers. System administrators looking at supporting Docker-based applications in production should consider diving deeper into the network configuration of the Docker engine and getting a good understanding of the defaults being set as well as how the networking namespaces are being used. Finally, for production use across multiple hosts, you should check the recipes about Weave and Flannel as well as start learning Docker network.

3.1 Finding the IP Address of a Container

Problem

You have started a container and would like to find its IP address.

Solution

There are many ways to find the IP address of a container started with the default Docker networking. This recipe presents a few of them.

The first method is to use the `docker inspect` command (see [Recipe 9.1](#) for details) and the Go template format:

```
$ docker run -d --name nginx nginx
$ docker inspect --format '{{ .NetworkSettings.IPAddress }}' nginx
172.17.0.2
```

You can also use the `docker exec` command and check the IP address by using a command that executes within the container:

```
$ docker exec -ti nginx ip add | grep global
    inet 172.17.0.2/16 scope global eth0
```

You could also check the `/etc/hosts` file in the container, assuming the image does set it properly:

```
$ docker run -d --name foobar -h foobar busybox sleep 300
$ docker exec -ti foobar cat /etc/hosts | grep foobar
172.17.0.4          foobar
```

Finally, you can enter a shell in the container and issue standard Linux commands at the prompt:

```
$ docker exec -ti nginx bash
root@a3c1f7edb00a:/# cat /etc/hosts
```

See Also

- To understand how Docker networking works and go beyond finding a container IP address, see [Recipe 3.4](#)
- 10 [examples](#) of how to get a Docker container IP address

3.2 Exposing a Container Port on the Host

Problem

You want to access a service running in a container over the network.

Solution

Docker can dynamically map a network port in a container to a port on the host by using the `-P` option of `docker run`. You can also manually specify a mapping by using the `-p` option.

Let's say that you have built an image that runs a Python Flask application using the Dockerfile shown here:

```
FROM python:2.7.10

RUN pip install flask
COPY hello.py /tmp/hello.py

CMD ["python", "/tmp/hello.py"]
```

This is similar to what you saw in [Recipe 2.4](#). Let's build this image and run a container without any port mapping flags:

```
$ docker build -t flask .
$ docker run -d --name foobar flask
```

You can find the IP address and reach the Flask application on port 5000 from within the host like so:

```
$ docker inspect -f '{{.NetworkSettings.IPAddress}}' foobar
172.17.0.2
$ curl http://172.17.0.2:5000/hi
Hello World!
```

However, you cannot reach this application from *outside* the host. To make this work, you are going to run the container again but this time using port mapping:

```
$ docker kill foobar
$ docker rm foobar
$ docker run -d -p 5000 --name foobar flask
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	...	PORTS	NA
2cc258827b34	flask	"python /tmp/hello.p	...	0.0.0.0:32768->5000/tcp	fo

You see that the `PORTS` column of `docker ps` now returns a mapping between port 32768 and port 5000 of the container. The host listens on interface 0.0.0.0, TCP port 32768 and forwards the requests to port 5000 of the container. Try to `curl` the Docker host on port 32768, and you will see that you reach the Flask application.

TIP

While `docker ps` returns the port-mapping information and you can use `docker inspect`, Docker also has the useful command `docker port` to list only the port mappings of a container. Try this:

```
$ docker port foobar 5000
0.0.0.0:32768
```

You might have noticed that your Dockerfile does not contain an `EXPOSE` statement as it did in [Recipe 2.4](#). If you add this statement, you can use the `-P` flag to expose the port and you do not have to specify the application port. Docker will automatically set the proper mapping. Add the `EXPOSE 5000` statement to the Dockerfile, build a new image, and run the container like so:

```
$ docker run -d -P flask
```

You will see that the mapping was done automatically.

Discussion

You can expose multiple container ports and choose TCP or UDP protocols; for example, if you wanted to expose port 5000 over TCP and port 53 over UDP (assuming your application uses port 53 as well), you would do this:

```
$ docker run -d -p 5000/tcp -p 53/udp flask
```

This port mapping is made possible via two mechanisms.

First, by default Docker can manipulate the IP table of the host. If you check your IP tables rules when running the preceding Flask application, you will find a new rule in the Docker chain:

```
$ sudo iptables -L
...

Chain DOCKER (1 references)
target      prot opt source                destination          tcp dpt:5000
ACCEPT      tcp  --  anywhere              172.17.0.2
```

Second, Docker starts a small proxy on your host that listens on the host interface using the port that was chosen dynamically. If you list the pro-

cesses, you will find this:

```
$ ps -ef | grep docker
root      29969      1   ... /usr/bin/docker -d
root      30851  29969   ... docker-proxy -proto tcp -host-ip 0.0.0.0 \
                                           -host-port 32769 \
                                           -container-ip 172.17.0.5
                                           -container-port 5000
```

You could change this default behavior by not allowing Docker to change your `iptables` but you will have to handle networking on your own (see [Recipe 3.6](#)).

3.3 Linking Containers in Docker

Problem

When building a distributed application that is made of several services, you need a way to discover where those services are so that various components of the system can reach the other ones. You could manually extract IP addresses of each service (running in a container), but in order to scale, you need a self-discovery system.

Solution

A first-order solution in Docker is to *link* containers. This is achieved by using the `--link` option of the `docker run` command.

WARNING

Container linking works well on a single host, but large-scale systems need other discovery mechanisms. Solutions like [Recipe 7.13](#) coupled with a key-value store and DNS might be chosen. Docker network (see [Recipe 3.14](#)) has a built-in mechanism that exposes container services without defining *links*.

To illustrate linking, let's build a three-tier system made of a database, a web application, and a load-balancer. You will start the database, link it to the web application container, and then start the load-balancer with a link to the web application. To ease linking, you will name each container. Because this is a toy example, where the application does not need to do anything, you will use the image `runseb/hostname`, a Flask application that returns the hostname of the container.

Let's start those three containers:

```

$ docker run -d --name database -e MYSQL_ROOT_PASSWORD=root mysql
$ docker run -d --link database:db --name web runseb/hostname
$ docker run -d --link web:application --name lb nginx
$ docker ps
CONTAINER ID   IMAGE                COMMAND                  ... PORTS                NAM
507edee2bbcf   nginx                "nginx -g 'daemon of   ... 80/tcp, 443/tcp        lb
62c321acb102   runseb/hostname     "python /tmp/hello     ... 5000/tcp              web
cf17b64e7017   mysql                "/entrypoint.sh mysq   ... 3306/tcp              dat

```

The result of linking is that the application container now contains environment variables that point to the database. Similarly, the load-balancer contains environment variables that point to the application container:

```

$ docker exec -ti web env | grep DB
DB_PORT=tcp://172.17.0.13:3306
DB_PORT_3306_TCP=tcp://172.17.0.13:3306
DB_PORT_3306_TCP_ADDR=172.17.0.13
DB_PORT_3306_TCP_PORT=3306
DB_PORT_3306_TCP_PROTO=tcp
DB_NAME=/web/db
DB_ENV_MYSQL_ROOT_PASSWORD=root
DB_ENV_MYSQL_MAJOR=5.6
DB_ENV_MYSQL_VERSION=5.6.25

$ docker exec -ti lb env | grep APPLICATION
APPLICATION_PORT=tcp://172.17.0.14:5000
APPLICATION_PORT_8080_TCP=tcp://172.17.0.14:5000
APPLICATION_PORT_8080_TCP_ADDR=172.17.0.14
APPLICATION_PORT_8080_TCP_PORT=5000
APPLICATION_PORT_8080_TCP_PROTO=tcp
APPLICATION_NAME=/lb/application

```

You can use these environment variables to dynamically configure your application and load-balancer.

The `/etc/hosts` file is also automatically updated to contain information for name resolution:

```

$ docker exec -ti web cat /etc/hosts
172.17.0.14      62c321acb102
172.17.0.13      db cf17b64e7017 database

$ docker exec -ti lb cat /etc/hosts
172.17.0.15      507edee2bbcf
172.17.0.14      application 62c321acb102 web

```

WARNING

If you restart a container, the `/etc/hosts` file of containers that were linked to it will be updated, but the environment variables will be unchanged. Therefore, using the content of `/etc/hosts` is the recommended way of extracting the IP address of a linked container.

Discussion

When you started the containers, you saw that we named all of them. The container name was used to define the link. It took this form:

```
--link <container_name>:<alias>
```

The alias specified was found in the `/etc/host` entry and was used as a prefix in the environment variables defined.

When you have containers running, you can use the `inspect` method to know what links are present. It will return a mapping between the linked container name and its alias in the container:

```
$ docker inspect -f "{{.HostConfig.Links}}" application
[/database:/application/db]
$ docker inspect -f "{{.HostConfig.Links}}" lb
[/application:/lb/app]
```

Although container linking is useful for development on a single machine, in large-scale deployments it will show its limitations, including when containers get restarted often. A system based on either DNS or a dynamic container registration system will scale and be updated automatically.

See Also

- Official documentation on [linking containers](#)

3.4 Understanding Docker Container Networking

Problem

You would like to understand the basics of networking Docker containers.

Solution

To use Docker and get network connectivity working between your containers, you don't have to use the solution presented here. It is presented in detail to provide enough information so that you can customize the networking behavior to your liking.

In the default installation of Docker, Linux bridge `docker0` is created on the Docker host. This bridge gets a private address and a subnet associated to it. The subnet assigned to the `docker0` bridge is chosen as the first nonconflicting subnet among the following list: `172.17.0.0/16`, `10.0.0.0/16`, `192.168.0.0/16`. Hence, most of the time your `docker0` bridge will get the address `172.17.0.1`. All containers that will attach to this bridge will get an address in the `172.17.0.0/24` network. Containers' networking interfaces get attached to this bridge and will use the `docker0` interface as a networking gateway. When a container is created, Docker creates a pair of "peer" network interfaces that are placed in two separate networking namespaces: one interface in the networking namespace of the container (i.e., `eth0`) and one interface in the networking namespace of the host, attached to the `docker0` bridge.

To illustrate this setup, let's look at a Docker host and start a container. You can use an existing Docker host or use the Vagrant box prepared for this book:

```
$ git clone https://github.com/how2dock/docbook
$ cd ch03/simple
$ vagrant up
$ vagrant ssh
```

Figure 3-1 shows the network configuration of this Vagrant box. It contains a single NAT interface to get to the outside network. Inside the host is Linux bridge `docker0`, and two containers are depicted.

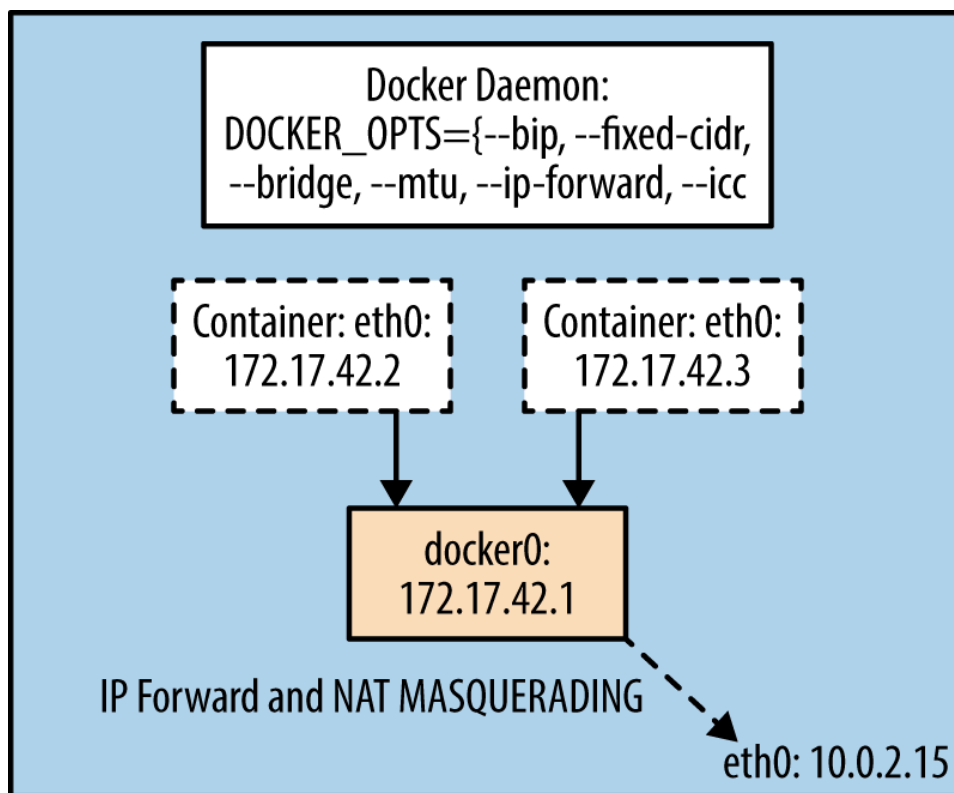


Figure 3-1. Network diagram of single Docker host

After connecting to the host, if you list the network links, you will see the loopback device, an `eth0` interface, and the `docker0` bridge, as depicted in the diagram. Docker starts when the machine is booted and automatically creates a bridge and assigns a subnet to it:

```
$ ip -d link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN ...
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 promiscuity 0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast ...
   link/ether 08:00:27:98:a7:ad brd ff:ff:ff:ff:ff:ff promiscuity 0
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue ...
   link/ether 00:00:00:00:00:00 brd ff:ff:ff:ff:ff:ff promiscuity 0
   bridge
```

Now let's start a container and check its network interface:

```
$ docker run -ti --rm ubuntu:14.04 bash
root@4e3ffb9bc381:/# ip addr show eth0
6: eth0: <BROADCAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group defa
   link/ether 02:42:ac:11:2a:03 brd ff:ff:ff:ff:ff:ff
   inet 172.17.42.3/24 scope global eth0
...
```

Indeed the container has an IP (i.e., 172.17.42.3) in the 172.17.42.1/16 network. On the host itself, a virtual interface (i.e., `veth450b81a` in the following code) is created and attached to the bridge. You can see this by listing the links on the Docker host with the `ip` tool (or `ifconfig`) and the `brctl` tool if you installed the `bridge-utils` package. The `ip` command

represents a collection of utilities for controlling TCP/IP traffic in Linux.

You can find the project documentation at the [Linux Foundation website](#).

```
$ ip -d link show
...
3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state
    link/ether aa:85:e0:61:69:2d brd ff:ff:ff:ff:ff:ff promiscuity 0
    bridge
7: veth450b81a: <BROADCAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast master do
    link/ether aa:85:e0:61:69:2d brd ff:ff:ff:ff:ff:ff promiscuity 1
    veth
$ brctl show
bridge name      bridge id                STP enabled    interfaces
docker0          8000.aa85e061692d        no             veth450b81a
```

From the container, you can ping the network gateway 172.17.42.1 (i.e., `docker0`), other containers on the same host, and the outside world.

TIP

Start another container on a separate terminal and try to ping each container. Verify that the second container interface is also attached to the bridge. Since there are no IP table rules dropping traffic, both containers can communicate with each other on any port.

Discussion

Outbound networking traffic is forwarded to the other interfaces of your Docker host via IP forwarding and will go through NAT translation using an IP table masquerading rule. On your Docker host you can check that IP forwarding has been enabled:

```
$ cat /proc/sys/net/ipv4/ip_forward
1
```

TIP

Try turning off IP forwarding, and you will see that your container will lose outbound network connectivity:

```
# echo 0 > /proc/sys/net/ipv4/ip_forward
```

You can also check the NAT rule that does the IP masquerading for outbound traffic:

```
$ sudo iptables -t nat -L
...
Chain POSTROUTING (policy ACCEPT)
target      prot opt source                destination
MASQUERADE  all  --  172.17.42.0/24          anywhere
...
```

In [Recipe 3.7](#), you will see how to create this configuration from scratch.

See Also

- Docker official networking [documentation](#)

3.5 Choosing a Container Networking Namespace

Problem

When starting a container, you want to be able to choose a specific network namespace. For certain applications that you run in containers, you may be required to use a different network setup than default bridge networking or you may not need any networking at all.

Solution

In [Recipe 3.4](#), you started a container using the defaults of the `docker run` command. This attached the container to a Linux bridge and created the appropriate network interfaces. It takes advantage of IP forwarding and IP tables managed by the Docker engine to provide outbound network connectivity and NAT.

However, you can start a container with a different type of networking. You can choose between the host networking namespace, no networking namespace at all, or the networking namespace of another container by using the `--net` option of `docker run`.

Let's start a container on a Docker host without any networking namespace by using `--net=none`:

```
$ docker run -it --rm --net=none ubuntu:14.04 bash
root@3a22f5076f9a:/# ip -d link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DE
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 promiscuity 0
root@3a22f5076f9a:/# route
```

Kernel IP routing table					
Destination	Gateway	Genmask	Flags	Metric	Ref
Use Ifa					

When listing the networking links, you see only a link local address. There are no other network interfaces and no networking routes. You will have to bring up the network manually if you need it (see [Recipe 3.7](#)).

Now let’s start a container with the networking namespace of the host by using `--net=host` :

```
$ docker run -it --rm --net=host ubuntu:14.04 bash
root@foobar-server:/# ip -d link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DE
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 promiscuity 0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state
    link/ether 08:00:27:98:a7:ad brd ff:ff:ff:ff:ff:ff promiscuity 0
3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state
    link/ether c6:4b:6b:b7:4b:98 brd ff:ff:ff:ff:ff:ff promiscuity 0
    bridge
```

When listing the networking links within this container, you see exactly the same interfaces as in the host, including the `docker0` bridge. This means that while the container processes are isolated in their own name-space and resources are limited through cgroups, the network namespace of the container is the same as the one of the host. You see in the preced-ing example that the hostname of the container is the Docker host host-name (you cannot use the `-h` option to set a hostname when using the host networking stack). Note, however, that you will not be able to recon-figure the host network from such a container. For example, you cannot bring down interfaces:

```
root@foobar-server:/# ifconfig eth0 down
SIOCSIFFLAGS: Operation not permitted
```

Although host networking can be handy, it needs to be handled with lots of care.

WARNING

Starting a container with `--net=host` can be dangerous, especially if you start a privileged container with `--privileged=true`. Host networking will allow you to change your host network configuration from within the container. If you were to run an application as `root` in a privileged container started with `--net=host`, a vulnerability of your application would let an intruder control your Docker host networking entirely. However, it can also be useful for processes that need a lot of network I/O.

The final option is to use the network stack of another already running container. Let's start a container with the hostname *cookbook*:

```
$ docker run -it --rm -h cookbook ubuntu:14.04 bash
root@cookbook:/# ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:ac:11:00:02
          inet addr:172.17.0.2  Bcast:0.0.0.0  Mask:255.255.0.0
          ...
```

You see at the prompt that the hostname has been set to *cookbook* and that the IP is 172.17.02. It is attached to the `docker0` bridge. Now let's start another container and use the same network namespace. First you list the running containers to get the name of the container just started. The convention is to use `--net=container:CONTAINER_NAME_OR_ID`:

```
$ docker ps
CONTAINER ID    IMAGE           COMMAND         ...    NAMES
cc7f72826c36    ubuntu:14.04    "bash"         ...    cocky_galileo
$ docker run -ti --rm --net=container:cocky_galileo ubuntu:14.04 bash
root@cookbook:/# ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:ac:11:00:02
          inet addr:172.17.0.2  Bcast:0.0.0.0  Mask:255.255.0.0
          ...
```

As you see, the new container has the same hostname as the first container started and of course has the same IP. The processes in each container will be isolated and exist in their own process namespace, but they share the same networking namespace and can communicate on the loopback device.

Discussion

Which networking namespace to use is up to the application you are running and what you want the network to look like. Docker networking is extremely flexible and will allow you to build any topology and secure network scenarios between your container processes.

See Also

- How Docker networks [containers](#)

3.6 Configuring the Docker Daemon IP

Tables and IP Forwarding Settings

Problem

You may not like that by default the Docker daemon turned on IP forwarding as well as modified your IP tables. You would like more control on how traffic flows on your host, between your containers and with the outside world.

Solution

The default Docker behavior will most likely be fine for most readers. However, this behavior is customizable when you start the Docker daemon with the `--ip-forward=false`, `--iptables=false` options. This recipe shows you to make those customizations.

To try this, stop the Docker daemon on the host that you are using. On Ubuntu/Debian-based systems, edit `/etc/default/docker` and set these options to `false` (on CentOS/RHEL systems edit `/etc/sysconfig/docker`):

```
$ sudo service docker stop
$ sudo su
# echo DOCKER_OPTS="--iptables=false --ip-forward=false" >> /etc/default/
# service docker restart
```

WARNING

You may have to remove the postrouting rule manually first as well as set the IP forwarding rule to zero, before restarting the Docker daemon. To do this, try the following on your Docker host:

```
# iptables -t nat -D POSTROUTING 1
# echo 0 > /proc/sys/net/ipv4/ip_forward
# service docker restart
```

With this configuration, traffic on the Docker bridge `docker0` will not be forwarded to the other networking interfaces and the postrouting masquerading rule will not be present. This means that all outbound connectivity from your containers to the outside world will be dropped.

Verify this behavior by starting a container and trying to reach the outside world. For example:

```
$ docker run -it --rm ubuntu:14.04 bash
WARNING: IPv4 forwarding is disabled.
```

```
root@ba12d578e6c8:/# ping -c 2 -W 5 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1009ms
```

To re-enable communication to the outside manually, enable IP forwarding and set the postrouting rule on the Docker host like so:

```
# echo 1 > /proc/sys/net/ipv4/ip_forward
# iptables -t nat -A POSTROUTING -s 172.17.0.0/16 -j MASQUERADE
```

Go back to the terminal of your container and try pinging `8.8.8.8` again. Traffic should now be routed outside your host.

WARNING

With `--iptables=false` set for the Docker daemon, you will not be able to restrict traffic between containers (i.e., use `--icc=false`) since Docker will not be able to manage the IP table rules. This means that all containers started on the same bridge will be able to communicate on all ports. See the following Discussion for more on this topic.

Discussion

By default the Docker daemon is allowed to manage the IP table rules on the Docker host. This means that it can add rules that restrict traffic between containers and provide network isolation between them.

If you disallow Docker to manipulate the IP table rules, it will not be able to add rules that restrict traffic between containers.



`-icc=false` option for the Docker daemon. This will add a default drop rule for all packets on the bridge, and containers will not be able to reach each other.

You can try this by editing the Docker config file (i.e., `/etc/default/docker` on Ubuntu/Debian and `/etc/sysconfig/docker` on CentOS/RHEL) and adding the `--icc=false` option. Restart Docker and start two containers on your host, and you will see that you cannot ping one container from another.

Since this drastically restricts traffic between containers, how can you have them communicating? This is solved with container linking, which

creates specific IP table rules (see [Recipe 3.3](#)).

Allow ping from the Docker host to all the containers:

```
$ sudo iptables -A DOCKER -p icmp --icmp-type echo-request -j ACCEPT
$ sudo iptables -A DOCKER -p icmp --icmp-type echo-reply -j ACCEPT
```

3.7 Using pipework to Understand Container Networking

Problem

Docker built-in networking capabilities work great, but you would like a hands-on approach that enables you to use traditional networking tools to create network interfaces for your containers.

Solution

This is an advanced recipe aimed at providing more in-depth knowledge of how Docker networking happens. You do not need this recipe and the tooling presented here to use Docker. However, to better understand Docker networking, you might want to use [pipework](#). Pipework, created by Jerome Petazzoni from Docker back in 2013, manipulates cgroups and network namespaces to build networking scenarios for your containers. At first it supported pure LXC containers and now it also supports Docker containers. If you start a container with the `--net=none` option, pipework is handy for adding networking to that container. This is a really nice exercise if you want to gain more detailed knowledge about Docker networking, although it's not needed for day-to-day use and production deployment.

NOTE

Using pipework is not needed to use Docker and manage the connectivity of your containers. This recipe is included for those who want to gain advanced knowledge in creating network stacks in the container network namespaces by hand. pipework allows you to do this, and having a look at the bash script gives you even more knowledge, as you will learn all the detailed step-by-step commands that are needed to build the network of a container.

While almost everything you can do with pipework is built in within Docker, it is a great tool to reverse-engineer Docker networking and get a deeper understanding of how the containers communicate with each other and the outside world. This recipe shows you a few examples so you

can deconstruct Docker networking capabilities and become a little more comfortable dealing with different networking namespaces.

pipework is a single bash script that you can [download for free](#). For convenience, I created a Vagrant box that contains pipework. You can get it by cloning the repository and starting the Vagrant VM:

```
$ git clone https://github.com/how2dock/docbook
$ cd ch03/simple
$ vagrant up
$ vagrant ssh
vagrant@foobar-server:~$ cd /vagrant
vagrant@foobar-server:/vagrant$ ls
pipework  Vagrantfile
```

Let's start a container without any network by using `--net=none` as shown in [Recipe 3.5](#):

```
$ docker run -it --rm --net none --name cookbook ubuntu:14.04 bash
root@556d04d8637e:/# ip -d link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode ..
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 promiscuity 0
```

In another terminal on the Docker host, let's use pipework to create a bridge `br0`, assign an IP address to the container, and set the correct routing from the container to the bridge:

```
$ cd /vagrant
$ sudo ./pipework br0 cookbook 192.168.1.10/24@192.168.1.254
Warning: arping not found; interface may not be immediately reachable
```

In the container, verify that the interface `eth1` is up and that the routing is in place:

```
root@556d04d8637e:/# ip -d link show eth1
7: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state
    link/ether a6:95:12:b9:8f:55 brd ff:ff:ff:ff:ff:ff promiscuity 0
    veth
root@556d04d8637e:/# route
Kernel IP routing table
```

Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
default	192.168.1.254	0.0.0.0	UG	0	0	0	eth
192.168.1.0	*	255.255.255.0	U	0	0	0	eth

Now if you list the network links on the host, you will see a bridge `br0` in addition to the default `docker0` bridge, and if you list the bridges (using

`brctl` from the *bridge-utils* package), you will see the virtual Ethernet interface attached to `br0` by `pipework`:

```
$ ip -d link show
...
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default
    link/ether 00:00:00:00:00:00 brd ff:ff:ff:ff:ff:ff promiscuity 0
    bridge
8: br0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state \ DOWN mode DEFAULT group default
    link/ether 22:43:24:f5:91:7e brd ff:ff:ff:ff:ff:ff promiscuity 0
    bridge
10: veth1pl31668: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc \ pfifo_fast master br0 state DOWN mode DEFAULT group default qlen 1000
    link/ether 22:43:24:f5:91:7e brd ff:ff:ff:ff:ff:ff promiscuity 1
    veth
$ brctl show
bridge name      bridge id                STP enabled  interfaces
br0              8000.224324f5917e        no          veth1pl31668
docker0          8000.000000000000        no
```

At this stage, you can reach the container from the host or reach any other containers from the container `cookbook`. However, if you try to reach outside the Docker host, you will notice that it will not work. There is no NAT masquerading rule in place that is added automatically by Docker when you use the defaults. Add the rule manually on the Docker host and try to ping `8.8.8.8` (for example) from the container interactive terminal:

```
# iptables -t nat -A POSTROUTING -s 192.168.0.0/16 -j MASQUERADE
```

On the container, verify that you can reach outside your Docker host:

```
root@556d04d8637e:/# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=61 time=22.6 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=61 time=23.8 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=61 time=23.9 ms
```

`pipework` can do a lot more, so make sure to check the *README file* and to pick inside the bash script to gain an even greater understanding of the networking namespace.

Discussion

Although pipework is extremely powerful and allowed you to build a proper networking stack for a container started with `--net=none`, it also hid some of the details of manipulating the container network namespace. If you read the code of pipework, you will see what it does. A good explanation is also available in the Docker [documentation](#) and is a good exercise, both in networking and containers. I highly recommend reading it.

NOTE

This discussion is not about pipework specifically; it aims to show you all the steps necessary to build a networking stack for a container. It is extremely useful to obtain a better understanding of container networking and reverse-engineer how Docker works.

Let's look back at this single pipework command:

```
$ sudo ./pipework br0 cookbook 192.168.1.10/24@192.168.1.254
```

This command did several almost magical things at once:

- It created a bridge `br0` on the host.
- It assigned IP address `192.168.1.254`.
- It created a network interface inside the container and assigned it IP address `192.168.1.10`.
- Finally it added a route inside the container, setting up the bridge as the default gateway.

Let's do it step by step but without pipework this time. To get started, let's add a bridge `br0` and give it the IP `192.168.1.254`. If you have worked on virtual machine virtualization before, this should be familiar. If not, follow along: you create the bridge with the `brctl` utility, use `ip` to add the IP address to the bridge, and finish by bringing the bridge up.

If you have followed along, you might want to delete the existing `br0` bridge:

```
$ sudo ip link set br0 down
$ sudo brctl delbr br0
```

You are now ready to do it all over again, but by hand this time:

```
$ sudo brctl addbr br0
$ sudo ip addr add 192.168.1.254/24 dev br0
$ sudo ip link set dev br0 up
```

The tricky part compared to full network virtualization comes from the fact that you are dealing with containers and that the networking stack is in fact a different network namespace on the host. To assign network interfaces to the container, you need to assign an interface to the network namespace that the container will use. The interfaces that you can assign to a specific network namespace are virtual Ethernet interface pairs. These pairs act as a pipe, with one end of the pipe in the container namespace and the other end on the bridge that you just created on the host.

Therefore, let's create a `veth` pair *foo*, *bar* and attach *foo* to the bridge `br0`:

```
$ sudo ip link add foo type veth peer name bar
$ sudo brctl addif br0 foo
$ sudo ip link set foo up
```

The result can be seen with `ip -d link show`; a new bridge `br0` and `foo` interface of type `veth` attached to it:

```
$ ip -d link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DE
group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 promiscuity 0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state
UNKNOWN mode DEFAULT group default qlen 1000
    link/ether 08:00:27:98:a7:ad brd ff:ff:ff:ff:ff:ff promiscuity 0
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue stat
DOWN mode DEFAULT group default
    link/ether 00:00:00:00:00:00 brd ff:ff:ff:ff:ff:ff promiscuity 0
    bridge
6: br0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP m
DEFAULT group default
    link/ether ee:7d:7e:f7:6f:18 brd ff:ff:ff:ff:ff:ff promiscuity 0
    bridge
8: foo: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast master
state UP mode DEFAULT group default qlen 1000
    link/ether ee:7d:7e:f7:6f:18 brd ff:ff:ff:ff:ff:ff promiscuity 1
    veth
$ brctl show
bridge name      bridge id                STP enabled    interfaces
br0               8000.ee7d7ef76f18        no             foo
docker0          8000.000000000000        not
```

TIP

Do not call each end of your veth pair the traditional `eth0` or `eth1` as it could conflict with existing physical interfaces on the host.

To complicate things a bit, when you started your container with `--net=none`, it did create a network namespace but there was nothing in it except the loopback device. Now that you want to configure it (e.g., adding an interface, setting up a route), you need to find the network namespace ID. Docker keeps its network namespaces in `/var/run/docker/netns`, which is a nondefault location. To be able to use the `ip` tool properly, you are going to do a little nonconventional hack and symlink `/var/run/docker/netns` to `/var/run/netns`, which is the default location where the `ip` tool looks for network namespaces. Doing so, you can list the existing network namespaces. Next, you see that the namespace ID of your container is the container ID:

```
$ cd /var/run
$ sudo ln -s /var/run/docker/netns netns
$ sudo ip netns
c785553b22a1
$ NID=$(sudo ip netns)
```

Now, let's put the *bar* veth in the container namespace by using `ip link set netns` and use `ip netns exec` to give it a name and a MAC address inside this namespace:

```
$ sudo ip link set bar netns $NID
$ sudo ip netns exec $NID ip link set dev bar name eth1
$ sudo ip netns exec $NID ip link set eth1 address 12:34:56:78:9a:bc
$ sudo ip netns exec $NID ip link set eth1 up
```

The final thing to do is to assign an IP address to *eth1* of the container and define a default route so that the container can reach the Docker host and beyond:

```
$ sudo ip netns exec $NID ip addr add 192.168.1.1/24 dev eth1
$ sudo ip netns exec $NID ip route add default via 192.168.1.254
```

That's it. At this stage, your container should have the exact same networking stack as the one built with `pipework` earlier with a single command.

TIP

Remember that if you want to reach outside your container, you need to add the IP NAT masquerading rule:

```
$ sudo iptables -t nat -A POSTROUTING -s 192.168.0.0/24
-j MASQUERADE
```

See Also

- [pipework's](#) extensive *README* covers multiple scenarios
- How Docker [networks](#) containers
- `ip netns` man [page](#)
- Introduction to Linux network [namespace](#)

3.8 Setting Up a Custom Bridge for Docker

Problem

You would like to set up your own bridge for Docker to use instead of using the default.

Solution

Create a bridge and change the start-up options of the Docker daemon to use it.

In the [Recipe 3.7](#) solution section, you saw how to create a full networking stack for a container started with the `--net=none` option. That section showed how to create a bridge. Let's reuse what we discussed there.

First let's turn off the Docker daemon, delete the `docker0` bridge created by default, and create a new bridge called `cookbook`:

```
$ sudo service docker stop
$ sudo ip link set docker0 down
$ sudo brctl delbr docker0
$ sudo brctl addbr cookbook
$ sudo ip link set cookbook up
$ sudo ip addr add 10.0.0.1/24 dev cookbook
```

Now that the bridge is up, you can edit the Docker daemon configuration file and restart the daemon (e.g., on Ubuntu):

```
$ sudo su
# echo 'DOCKER_OPTS="-b=cookbook"' >> /etc/default/docker
# service docker restart
```

You can start a container and list the IP address assigned to it and check network connectivity:

```
root@c557cdb072ba:/# ip addr show eth0
10: eth0: <BROADCAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group def
    link/ether 02:42:0a:00:00:02 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.2/24 scope global eth0
        ...
```

Automatically as expected, Docker also creates the NAT rule for this bridge:

```
$ sudo iptables -t nat -L
...
Chain POSTROUTING (policy ACCEPT)
target     prot opt source                destination
MASQUERADE all  --  10.0.0.0/24            anywhere
```

Discussion

Although you can do this manually, there is nothing different between the bridge `cookbook` that you just created and the default `docker0` bridge.

If you wanted to change the IP range that Docker uses for the containers started with the default networking (i.e., bridge), you could use the `--bip` option. You could also restrict this IP range with the `--fixed-cidr` option as well as set the MTU size with `--mtu`.

To bring down the bridge, execute the following two commands:

```
$ sudo ip link set cookbook down
$ sudo brctl delbr cookbook
```

3.9 Using OVS with Docker

Problem

You know how to use your own bridge to network your Docker containers (see [Recipe 3.8](#)), but you would like to use the Open vSwitch (OVS) virtual switch instead of the standard Linux bridge. Maybe you want to

build your own GRE, or VXLAN-based overlay, or you want to build a software-defined network solution with a network controller. OVS provides programmatic extensions and control using the OpenFlow [protocol](#) and the OVSDB management [protocol](#).

Solution

WARNING

As of Docker 1.7, Open vSwitch is not yet supported natively. You can use it, but you will need to use a tool like [pipework](#) (see [Recipe 3.7](#)) or a utility called [ovs-docker](#), a manual process to build the network stack of the containers. It should be supported in a future version of Docker Network (see [Recipe 3.14](#)).

Use [Open vSwitch](#) (OVS) as your bridge and specify its name in the Docker daemon configuration file.

On your Docker host, start by installing the packages for OVS; for example, on Ubuntu 14.04:

```
$ sudo apt-get -y install openvswitch-switch
```

TIP

If you want a more recent version of Open vSwitch, you can build it from [source](#) relatively easily.

Now create a bridge and bring it up:

```
$ sudo ovs-vsctl add-br ovs-cookbook
$ sudo ip link set ovs-cookbook up
```

You are now ready to use [pipework](#) (see [Recipe 3.7](#)) to build the network stack of containers attached to this Open vSwitch bridge. You will need to start containers without a network stack (i.e., `--net=none`)—for example:

```
$ docker run -it --rm --name foobar --net=none ubuntu:14.04 bash
root@8fda6e33eb88:/#
```

In a different shell on your Docker host, use [pipework](#) to create a network interface (you will need to install [pipework](#)) in your *foobar* container:

```
$ sudo su
# ./pipework ovs-cookbook foobar 10.0.0.10/24@10.0.0.1
# ovs-vsctl list-ports ovs-cookbook
veth1p131350
```

Your bridge will also have been assigned the IP address 10.0.0.1 by pipework:

```
$ ifconfig
ovs-cookbook  Link encap:Ethernet  HWaddr 36:b1:d3:e5:fc:44
               inet addr:10.0.0.1  Bcast:0.0.0.0  Mask:255.255.255.0
               ...
```

Your container will now have a network interface:

```
root@8fda6e33eb88:/# ifconfig
eth1          Link encap:Ethernet  HWaddr 52:fe:9f:78:b7:fc
               inet addr:10.0.0.10  Bcast:0.0.0.0  Mask:255.255.255.04
               ....
```

You could also create the interface by hand by using `ip netns`, as you did in the discussion section of [Recipe 3.7](#).

See Also

- The Open vSwitch [website](#)

3.10 Building a GRE Tunnel Between Docker Hosts

Problem

You need to have network connectivity between containers on multiple hosts using their own IP addresses.

Solution

Several recipes (see [Recipe 3.11](#) and [Recipe 3.13](#)) show solutions that can be used in production deployment. This recipe presents a step-by-step example of how to set up multihost networking, to help you build a solid foundation for understanding Docker networking.

To build a toy setup that provides network connectivity between containers on multiple hosts, build a Generic Routing Encapsulation (GRE) tunnel to encapsulate IPv4 traffic and provide a route between containers using their private addresses. To showcase this technique, you are going to bring up two Docker hosts and set up the network configuration that you can see in [Figure 3-2](#).

Host 1 has IP address `192.168.33.11`. We will give the `docker0` bridge IP address `172.17.0.1` and create a GRE tunnel endpoint with IP address `172.17.0.2`. Docker will give containers addresses in the `172.17.0.0/17` network.

Host 2 has IP address `192.168.33.12`. We will give the `docker0` bridge IP address `172.17.128.1` and create a GRE tunnel endpoint with IP address `171.17.128.2`. Docker will give containers addresses in the `172.17.128.0/17` network.

Splitting a `/16` network in two `/17` networks and assigning each subnet to the two different hosts ensures that containers will not get conflicting IP addresses.

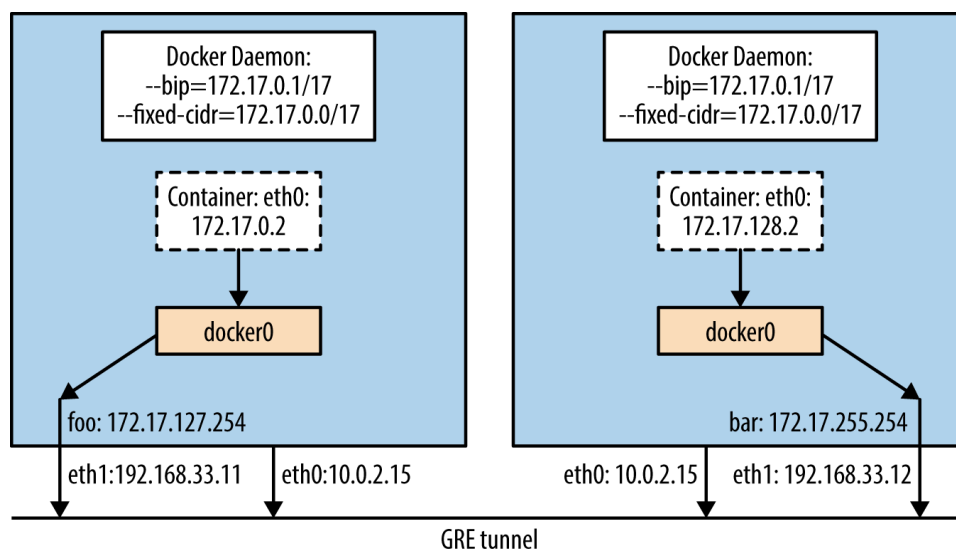


Figure 3-2. Network diagram of a two-hosts GRE tunnel overlay

You can start this configuration with a [Vagrantfile](#). Each host has the latest stable version of Docker and two network interfaces: one NAT interface that gives outbound connectivity and one interface on a private network.

The first thing to do to avoid any issues is to stop the Docker engine and remove that `docker0` bridge that was started during the Docker provisioning step. You will need to do this on all your hosts:

```
$ sudo su
# service docker stop
```

```
# ip link set docker0 down
# ip link del docker0
```

Now you can create a GRE tunnel between the two hosts. You do not need Open vSwitch for this; you can just use the `ip` tool. If you used the Vagrantfile mentioned earlier, on your first host with IP `192.168.33.11` do the following:

```
# ip tunnel add foo mode gre local 192.168.33.11 remote 192.168.33.12
# ip link set foo up
# ip addr add 172.17.127.254 dev foo
# ip route add 172.17.128.0/17 dev foo
```

If you did not use the Vagrantfile mentioned, replace the IP addresses for the `local` and `remote` endpoints in the preceding `ip tunnel` command with the IP addresses of your two Docker hosts. In the previous four commands, you created a GRE tunnel named `foo`. You brought the interface up and assigned an IP address to it. Then you set up a route that sends all `172.17.128.0/17` traffic in the tunnel.

On your second host, repeat the previous step to create the other end of the tunnel. You call this other end `bar` and set up a route that sends all `172.17.0.0/17` traffic over this tunnel:

```
# ip tunnel add bar mode gre local 192.168.33.12 remote 192.168.33.11
# ip link set bar up
# ip addr add 172.17.255.254 dev bar
# ip route add 172.17.0.0/17 dev bar
```

Once the tunnel is up, verify that you can ping back and forth using the tunnel. Now let's bring up Docker on both hosts. First you need to configure each Docker daemon so that it uses the appropriate subnets for its containers and uses the correct IP address for the `docker0` bridge. To do this, you edit the Docker daemon configuration file and use the `--bip` and `--fixed-cidr` options.

On host 1 that would be as follows:

```
# echo DOCKER_OPTS="--bip=172.17.0.1/17 --fixed-cidr=172.17.0.0/17" \
>> /etc/default/docker
```

And on host 2:

```
# echo DOCKER_OPTS="--bip=172.17.128.1/17 --fixed-cidr=172.17.128.0/17" \
>> /etc/default/docker
```

If you have chosen a different partitioning schema or have more than two hosts, repeat this with your values.

NOTE

Since Docker will turn on IP forwarding, all traffic on `docker0` will get forwarded to `foo` and `bar`, so there is no need to attach the tunnel endpoints to any bridges.

All that is left now is to restart Docker. Then you can start one container on each host and you will see that they have direct network connectivity using the private IP address given to them by Docker.

Discussion

There are multiple ways to build a networking overlay for your Docker host. Docker Network (see [Recipe 3.14](#)), which should be released in Docker 1.8, allows you to build VXLAN overlays using Docker built-in features. Other third-party solutions exist, such as Weave (see [Recipe 3.11](#)) or Flannel (see [Recipe 3.13](#)). As the Docker plug-in framework matures, this type of functionality will change significantly. For instance, Weave and Flannel will be available as Docker plug-ins, instead of a separate network setup.

See Also

- The blog post from Vincent Viallet on [Wiredcraft](#) that inspired this recipe

3.11 Running Containers on a Weave Network

Contributed by Fintan Ryan

Problem

You wish to create a network for your containers that scales from a single host to thousands of hosts across multiple data centers, with automatic IP address allocation and integrated service discovery via DNS.

Solution

Use [Weave Net](#) from [Weaveworks](#).

To help you experiment with Weave Net, I have created a Vagrantfile that starts two hosts running Ubuntu 14.04, and installs Docker, Weave Net, and two example containers. You can test it as follows:

```
$ git clone https://github.com/how2dock/docbook.git
$ cd ch03/weavesimple
$ vagrant up
```

Here are our Vagrant hosts in this example:

- 172.17.8.101 weave-gs-01
- 172.17.8.102 weave-gs-02

Next you will launch Weave Net on both hosts. Notice that you provide the IP address of the first host when launching Weave Net on the second host:

```
$ vagrant ssh weave-gs-01
$ weave launch
$ vagrant ssh weave-gs-02
$ weave launch 172.17.8.101
```

At this point, you have created a network that will automatically allocate IP addresses to any container launched with Weave Net, and integrated service discovery with DNS.

Containers that you launch that are aware of Weave Net will be automatically allocated unique IP addresses, and, where an `-h` option is provided to Docker, registered with DNS.

Next you will launch containers on each host. To allow you to easily launch containers on your network, you set your `DOCKER_HOST` environment variable using `weave env`:

```
$ vagrant ssh weave-gs-01
$ eval $(weave env)
$ docker run -d -h lb.weave.local fintanr/myip-scratch
$ docker run -d -h lb.weave.local fintanr/myip-scratch
$ docker run -d -h hello.weave.local fintanr/weave-gs-simple-hw

$ vagrant ssh weave-gs-02
$ eval $(weave env)
$ docker run -d -h lb.weave.local fintanr/myip-scratch
$ docker run -d -h hello-host2.weave.local fintanr/weave-gs-simple-hw
```

You have done two things here. First, you have launched a container with a simple Hello World application on your hosts. Second, you have used

DNS to create a load-balanced service across the containers named *lb*.

Let's launch a container on your Weave network and make some requests to the various containers you have launched:

```
$ vagrant ssh weave-gs-01
$ eval $(weave env)
$ C=$(docker run -d -ti fintanr/weave-gs-ubuntu-curl)
$ docker attach $C
root@ad6b7c0b1c6e:/#
root@ad6b7c0b1c6e:/# curl lb
Welcome to Weave, you probably want /myip
root@ad6b7c0b1c6e:/# curl lb/myip
10.128.0.2
root@ad6b7c0b1c6e:/# curl lb/myip
10.160.0.1
root@ad6b7c0b1c6e:/# curl hello
{
  "message" : "Hello World",
  "date" : "2015-07-09 15:59:50"
}
```

You also provide a script for the preceding commands, launching your container to make requests:

```
$ ./launch-simple-demo.sh
```

Discussion

Weave Net allows you to quickly and easily launch containers on a scalable network with automatic IP address allocation and service discovery.

In this example, you launched a Weave router container on your first host, `weave-gs-01`. On your second host, `weave-gs-02`, you launched another Weave router container with the IP address of your first host. This command tells Weave on `weave-gs-02` to peer with the Weave router on `weave-gs-01`.

Any containers you launch after this using Weave are visible within the Weave network to all other containers, no matter what host they are on. The containers will be automatically allocated an IP address that is unique on the network, and automatically registered with the Weave DNS service if Docker is called with an `-h` option.

To examine the containers launched, you can also use Weave `Scope` (see [Recipe 9.12](#)). On each host, run:

```
$ scope launch
```

See Also

- [Weave Getting Started Guides](#)

3.12 Running a Weave Network on AWS

Contributed by Fintan Ryan

Problem

You would like to use Weave Net and WeaveDNS on instances deployed in AWS.

Solution

As prerequisites, you will need the following:

- An account on AWS
- A set of access and secret API keys
- Ansible installed, with the boto package

To help you experiment with Weave on AWS, I have created an Ansible playbook that starts two hosts running Ubuntu 14.04 on EC2, installs Docker, and installs Weave. I have provided a second playbook specifically for launching a simple demo application using HAProxy as a load-balancer in front of containers across your two hosts:

```
$ git clone https://github.com/how2dock/docbook.git
$ cd ch03/weaveaws
$ ansible-playbook setup-weave-ubuntu-aws.yml
```

TIP

You can change your AWS region and AMI in the file *ansible_aws_variables.ym*.

To launch your containers, call the following:

```
$ ansible-playbook launch-weave-haproxy-aws-demo.yml
```


I have provided a script to quickly connect to your HAProxy container and cycle through a number of requests. Each container will return its hostname as part of its JSON output:

```
$ ./access-aws-hosts.sh

Connecting to HAProxy with Weave on AWS demo

{
  "message" : "Hello Weave - HAProxy Example",
  "hostname" : ws1.weave.local",
  "date" : "2015-03-13 11:23:12"
}

{
  "message" : "Hello Weave - HAProxy Example",
  "hostname" : ws4.weave.local",
  "date" : "2015-03-13 11:23:12"
}

{
  "message" : "Hello Weave - HAProxy Example",
  "hostname" : ws5.weave.local",
  "date" : "2015-03-13 11:23:12"
}

....
```

Discussion

Using Weave Net, you have placed HAProxy as a load-balancing solution in front of a number of containers running a simple application distributed across a number of hosts.

See Also

- [Weave Getting Started Guides](#)

3.13 Deploying flannel Overlay Between Docker Hosts

Contributed by Eugene Yakubovich

Problem

You want containers on different hosts to communicate with each other without port mapping.

Solution

Use `flannel` to create an overlay network for containers. Each container will be assigned an IP that can be reachable from other hosts. Start by bringing up two virtual machines from a Vagrantfile:

```
$ git clone https://github.com/how2dock/docbook.git
$ cd ch03/flannel
$ vagrant up
```

This defines two virtual machines installed with Docker: `etcd` and `flannel`. The “master” will run a key-value store (`etcd`) that `flannel` uses for coordination.

Next, `vagrant ssh master` and start `etcd` in the background:

```
$ cd /opt/coreos/etcd-v2.0.13-linux-amd64
$ nohup ./etcd --listen-client-urls=http://0.0.0.0:2379 \
    --advertise-client-urls=http://192.168.33.10:2379 &
```

Before starting the `flannel` daemon, write the overlay network configuration into `etcd`. Be sure to pick a subnet range that does not conflict with other IP addresses:

```
$ ./etcdctl set /coreos.com/network/config '{ "Network": "10.100.0.0/16" }'
```

Now start the `flannel` daemon. Notice that the `--iface` option specifies the IP of the private network given in the Vagrantfile. `flannel` will forward encapsulated packets over this interface:

```
$ cd /opt/coreos/flannel-0.5.1
$ sudo ./flanneld --iface=192.168.33.10 --ip-masq &
$ sudo ./mk-docker-opts.sh -c -d /etc/default/docker
```

`flannel` will acquire a lease for a `/24` subnet to be assigned to the `docker0` bridge. The acquired subnet will be written out to the `/run/flannel/subnet.env` file. The `mk-docker-opts.sh` utility converts this file into a set of command-line options for the Docker daemon.

Finally, start the Docker daemon. Verify that everything is running as expected by checking the IP of the `docker0` bridge. It should be within the `10.100.0.0/16` range:

```
$ sudo service docker start
$ ifconfig docker0
docker0    Link encap:Ethernet    HWaddr 56:84:7a:fe:97:99
           inet addr:10.100.63.1    Bcast:0.0.0.0    Mask:255.255.255.0
...

```

Over on the “worker” node, repeat the procedure of bringing up `flannel`. Since `etcd` is running on the “master,” do not launch it on this node. Instead, point `flannel` to use the instance running on “master”:

```
$ cd /opt/coreos/flannel-0.5.1
$ sudo ./flanneld --etcd-endpoints=http://192.168.33.10:2379 \
               --iface=192.168.33.11 --ip-masq &
$ sudo ./mk-docker-opts.sh -c -d /etc/default/docker
$ sudo service docker start

```

With both nodes bootstrapped into the `flannel` network, bring up a simple busybox container on each of the nodes. The containers will have an IP pingable from the remote container.

Discussion

All `flannel` members use `etcd` for coordination. Upon start-up, the `flannel` daemon reads the overlay network configuration from `etcd` as well as all other subnets in use by other nodes. It then picks a used subnet (/24 by default) and attempts to claim it by creating a key for it in `etcd`. If the creation succeeds, the node has acquired a 24-hour lease on the subnet. The associated value contains the host’s IP.

Next, `flannel` uses the TUN device to create a `flannel0` interface. IP fragments routed to `flannel0` from the `docker0` bridge will be delivered to the `flannel` daemon. It encapsulates each IP fragment in a UDP packet and uses the subnet information from `etcd` to forward it to the correct host. The receiving end unwraps the IP fragment from its encapsulation and sends it to `docker0` via the TUN device.

`flannel` continues to watch `etcd` for changes in the memberships to keep its knowledge current. Additionally, the daemon will renew its lease an hour before its expiration.

3.14 Networking Containers on

Multiple Hosts with Docker Network

Problem

Although you could build tunnels between your Docker hosts manually (see [Recipe 3.10](#)), you want to take advantage of the new Docker Network feature and use a VXLAN overlay.

Solution

WARNING

Docker Network is a new feature, currently available on the Docker experimental channel. This recipe is a preview that will give you a taste of what you will be able to find in future Docker releases.

NOTE

As of this writing, Docker Network relies on Consul for key-value stores, uses Serf for discovery of the nodes, and builds a VXLAN overlay by using the standard Linux bridge. Since Docker Network is under heavy development, these requirements and methods may change in the near future.

As is common in this book, I prepared a Vagrantfile that will start three virtual machines. One will act as a Consul server, and the other two will act as Docker hosts. The experimental version of Docker is installed on the two Docker hosts, while the latest stable version of Docker is installed on the machine running the Consul server.

The setup is as follows:

- `consul-server`, the Consul server node based on Ubuntu 14.04, has IP 192.168.33.10.
- `net-1`, the first Docker host based on Ubuntu 14.10, has IP 192.168.33.11.
- `net-2`, the second Docker host based on Ubuntu 14.10, has IP 192.168.33.12.

[Figure 3-3](#) illustrates this setup.

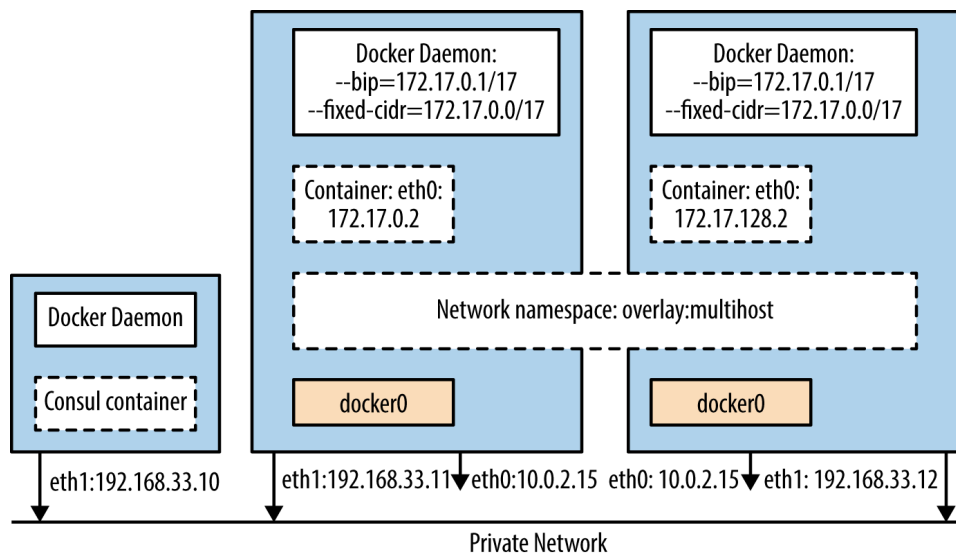


Figure 3-3. Network diagram of a two-hosts Docker network VXLAN overlay with an additional Consul node

Clone the repository, change to the *docbook/ch03/networks* directory, and let Vagrant do the work:

```
$ git clone https://github.com/how2dock/docbook/
$ cd docbook/ch03/network
$ vagrant up
$ vagrant status
Current machine states:

consul-server           running (virtualbox)
net-1                   running (virtualbox)
net-2                   running (virtualbox)
```

You are now ready to `ssh` to the Docker hosts and start containers. You will see that you are running the Docker experimental version `-dev`. The version number that you see may be different, depending on where you are in the release cycle:

```
$ vagrant ssh net-1
vagrant@net-1:~$ docker version
Client version: 1.9.0-dev
...<snip>...
```

Check that Docker Network is functional by listing the default networks:

```
vagrant@net-1:~$ docker network ls
NETWORK ID          NAME                TYPE
4275f8b3a821        none               null
80eba28ed4a7        host              host
64322973b4aa        bridge            bridge
```

No service has been published so far, so `docker service ls` will return an empty list:

```
$ docker service ls
```

SERVICE	ID	NAME	NETWORK	CONTAINER
---------	----	------	---------	-----------

Start a container and check the content of `/etc/hosts`:

```
$ docker run -it --rm ubuntu:14.04 bash
root@df479e660658:/# cat /etc/hosts
172.21.0.3      df479e660658
127.0.0.1      localhost
::1           localhost ip6-localhost ip6-loopback
fe00::0       ip6-localnet
ff00::0       ip6-mcastprefix
ff02::1       ip6-allnodes
ff02::2       ip6-allrouters
172.21.0.3     distracted_bohr
172.21.0.3     distracted_bohr.multihost
```

In a separate terminal on `net-1`, list the networks again. You will see that the `multihost` overlay now appears. The overlay network `multihost` is your default network. This was set up by the Docker daemon during the Vagrant provisioning. Check `/etc/default/docker` to see the options that were set:

```
vagrant@net-1:~$ docker network ls
```

NETWORK	ID	NAME	TYPE
4275f8b3a821		none	null
80eba28ed4a7		host	host
64322973b4aa		bridge	bridge
b5c9f05f1f8f		multihost	overlay

Now in a separate terminal, `ssh` to `net-2`, and check the network and services. The networks will be the same, and the default network will also be `multihost` of type `overlay`. But the service will show the container started on `net-1`:

```
$ vagrant ssh net-2
vagrant@net-2:~$ docker service ls
```

SERVICE	ID	NAME	NETWORK	CONTAINER
b00f2bfd81ac		distracted_bohr	multihost	df479e660658

Start a container on `net-2` and check `/etc/hosts`:

```
vagrant@net-2:~$ docker run -ti --rm ubuntu:14.04 bash
root@2ac726b4ce60:/# cat /etc/hosts
172.21.0.4          2ac726b4ce60
127.0.0.1          localhost
::1               localhost ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
172.21.0.3          distracted_bohr
172.21.0.3          distracted_bohr.multihost
172.21.0.4          modest_curie
172.21.0.4          modest_curie.multihost
```

You will see not only the container that you just started on `net-2`, but also the container that you started earlier on `net-1`. And, of course, you will be able to ping each container.

Discussion

The solution uses the default network overlay that was configured at start-up time by specifying it in the `/etc/default/docker` configuration file. You can, however, use a nondefault overlay network. This means that you can create as many overlays as you want, and the container started in each overlay will be isolated.

In the previous test, you started containers with regular options `-ti --rm`. These containers were placed automatically in the default network, which was set to be the `multihost` network of type `overlay`.

But you could create your own overlay network and start containers in it. Let's try this. First, create a new overlay network with the `docker network create` command.

On one of your Docker hosts, `net-1` or `net-2`, run the following:

```
$ docker network create -d overlay foobar
8805e22ad6e29cd7abb95597c91420fdcac54f33fcdd6fbca6dd4ec9710dd6a4
$ docker network ls
NETWORK ID          NAME                TYPE
a77e16a1e394        host                host
684a4bb4c471        bridge              bridge
8805e22ad6e2        foobar              overlay
b5c9f05f1f8f        multihost           overlay
67d5a33a2e54        none                null
```

Automatically, the second host will also see this network. To start a container on this new network, use the `--publish-service` option of

```
docker run:
```

```
$ docker run -it --rm --publish-service=bar.foobar.overlay ubuntu:14.04 bas
```

NOTE

You could directly start a container with a new overlay by using the `--publish-service` option, and it will create the network automatically.

Check the docker services now:

```
$ docker service ls
```

SERVICE	ID	NAME	NETWORK	CONTAINER
	blffdbfb1ac6	bar	foobar	6635a3822135

Repeat the getting started steps by starting another container in this new overlay on the other host, check the `/etc/hosts` file, and try to ping each container.

3.15 Diving Deeper into the Docker Network Namespaces Configuration

Problem

You would like to better understand what Docker Network (see [Recipe 3.14](#)) does, especially where the VXLAN interfaces exist.

Solution

The new Docker overlay networking in Docker Network is made possible via VXLAN tunnels and the use of network namespaces. In [Recipe 3.7](#), you already saw how to explore and manipulate network namespaces. The same can be done for Docker Network.

WARNING

This is an advanced recipe included so you can gain a deep understanding of how the network namespaces of a Docker host are used to build a VXLAN network overlay. This recipe is not needed to use default single-host Docker networking or multihost Docker Network or other multihost networking solutions like the ones presented in [Recipe 3.11](#) and [Recipe 3.13](#).

Check the [design documentation](#) for all the details. But to explore these concepts a bit, nothing beats an example.

Discussion

With a running container in one overlay, check the network namespace:

```
$ docker inspect -f '{{ .NetworkSettings.SandboxKey }}' 6635a3822135
/var/run/docker/netns/6635a3822135
```

This is not a default location for network namespaces, which might confuse things a bit. So let's become `root`, head over to this directory that contains the network namespaces of the containers, and check the interfaces:

```
$ sudo su
root@net-2:/home/vagrant# cd /var/run/docker/
root@net-2:/var/run/docker# ls netns
6635a3822135
8805e22ad6e2
```

To be able to check the interfaces in those network namespaces by using the `ip` command, create a symlink for `netns` that points to `/var/run/docker/netns`:

```
root@net-2:/var/run# ln -s /var/run/docker/netns netns
root@net-2:/var/run# ip netns show
6635a3822135
8805e22ad6e2
```

The two namespace IDs returned are the ones of the running container on that host and the overlay network the container is in:

```
root@net-2:/var/run/docker# ip netns exec 6635a3822135 ip addr show eth0
15: eth0: <BROADCAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group def
    link/ether 02:42:b3:91:22:c3 brd ff:ff:ff:ff:ff:ff
    inet 172.21.0.5/16 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::42:b3ff:fe91:22c3/64 scope link
        valid_lft forever preferred_lft forever
```

You get back the network interface of your running container, with the same MAC address, and the same IP. If you check the links of the overlay namespace, you see your VXLAN interface and the VLAN ID being used:

```
root@net-2:/var/run/docker# ip netns exec 8805e22ad6e2 ip -d link show
...<snip>...
14: vxlan1: <BROADCAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br0 state
UNKNOWN mode DEFAULT group default
    link/ether 7a:af:20:ee:e3:81 brd ff:ff:ff:ff:ff:ff promiscuity 1
    vxlan id 256 srcport 32768 61000 dstport 8472 proxy l2miss l3miss agein
    bridge_slave
16: veth2: <BROADCAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast master br0 sta
UP mode DEFAULT group default qlen 1000
    link/ether 46:b1:e2:5c:48:a8 brd ff:ff:ff:ff:ff:ff promiscuity 1
    veth
    bridge_slave
```

If you sniff packets on these interfaces, you will see the traffic between your containers.